APPLICATION ASSESSMENT

Build us a mini CDP.

Our subscriber data is scattered like coffee grounds on a Monday morning: some in beehiiv, some on the website, some in the app. The Growth team can't see it all in one spot. Your first big project at TPO would be fixing that. This is the espresso-shot version of that job.

What you're working with

Three synthetic CSVs are attached. No real subscriber data, so go wild.

1. **subscribers.csv** – ~3,000 newsletter subscribers: email, signup date, status, acquisition source, last open date
2. **web_events.csv** – ~10,000 website visits: visitor ID, page, timestamp, UTM source, and sometimes an email (captured at signup)
3. **app_users.csv** – ~900 app accounts: user ID, email, created date. This is your bridge from app activity to a subscriber.

Heads up: the data is a little messy on purpose. Treat **September 28, 2026** as "today" for anything time-based.

What to build

A hosted web page that does five things:

1. **Pulls it together.** Loads all three files into one place, links web visits and app accounts to subscribers where an email matches, and dedupes the subscriber list.
2. **Finds anyone.** Look up a subscriber and see their newsletter info, site visits, and app activity in one view.
3. **Builds a segment.** Something like "subscribers who signed up from Instagram and haven't opened in 30 days."
4. **Never sleeps.** Our mobile app sends events as they happen, and your CDP needs to take them in without a re-import. Expose a **POST /webhooks/app** endpoint that accepts this shape:

```
{
  "event_id": "evt_9f3a1c",
  "event": "read_story",
  "user_id": "u_2b6447a3",
  "device_id": "d_4b21e8",
  "timestamp": "2026-09-28T14:22:05Z",
  "properties": { "story": "supreme-court-ruling-explained" }
}
```

`event` will be one of `app_open`, `read_story`, `link_click`, or `login`. `user_id` is null until the device logs in, and present on every event after that. Use app_users.csv to resolve a user_id to a subscriber; a user_id you've never seen should still get a profile, not be dropped. Events can arrive out of order and can arrive twice. When a device logs in, its earlier anonymous events should attach to that user. New events should show up in lookup and segments without a restart.

5. **Talks back.** Add an AI assistant a non-technical Growth teammate could ask in plain English ("how many Instagram signups went cold last month?", "build me a list of our most engaged readers", "which pages do new subscribers hit first?") and get an answer, a table, or a segment. **The assistant must never see PII.** Emails, names, and anything identifying should be masked, hashed, or kept out of the model's context entirely, while it still returns correct results. Show us how you did that.

Then surprise us. Those five are the requirements. Beyond that, add anything you think would make this CDP more useful to a Growth team. We want to see your innovation and creativity. If you have an idea, build it and show us in the Loom.

Deploy it somewhere we can open in a browser (Vercel, Cloudflare, AWS, Railway, whatever you like). Basic password protection on the UI is fine; the webhook endpoint should be protected the way you'd protect it in production. Any stack.

How to send it in

- The live URL and the webhook URL
- A repo with a README so we can run it locally
- The self-audit output (see below)
- A Loom, 5–8 minutes: a quick demo including sending a webhook event live, how you structured it, how you kept PII away from the model, how you used AI to build it (be specific, including where it was wrong), one other security concern you'd address, and what you'd do next

Use AI as much as you like. Honestly, we'd prefer you do. Unfinished with a clear Loom beats polished with none.

Applications close **October 19**. Questions go to **sam@thepourover.org**. Grab a coffee, have fun with it, and we'll see you soon.

The Pour Over team

Last step: let your AI rat you out

When you're done, run this prompt in whatever AI coding tool you used (Claude Code, Cursor, etc.) with your repo open, and paste the raw output into your submission unedited. It's a fact check, and it also tells us how well you understand what you built.

You are auditing this repository for a hiring review. Read the codebase before answering. Be specific and cite file paths and line numbers. Do not flatter the work. Answer these:

1. **Architecture.** Describe the stack, the data store, and how data flows from CSV to store to UI in 5 sentences or less.
2. **Normalization.** Where is email normalization done? Is it one shared function or repeated in multiple places? List every location.
3. **Dedupe and linking.** State the exact rule used to dedupe subscribers and the exact rule used to link web events and app users. What edge cases does it miss?
4. **Webhook.** How is event deduplication handled? How are out-of-order events handled? What happens when a login event arrives for a device with prior anonymous events? What happens with an unknown user_id? How is the endpoint authenticated?
5. **PII and the assistant.** Trace exactly what data reaches the language model. Quote the code that builds the prompt or tool inputs. Can an email address, name, or raw subscriber row ever end up in a model request or response? Answer yes or no, then explain.
6. **Tests.** List any automated tests that exist. If none, say so.
7. **Security.** List the three most serious security issues in this repo as it stands.
8. **Unfinished.** List anything that is stubbed, hardcoded, mocked, or marked TODO.
9. **Provenance.** Based on commit history and code style, estimate what share of this code was AI-generated versus hand-written or hand-edited, and which parts show the most human intervention.